NETBRIDGE

Offline payment research prototype

A browser-based study of payment handoffs when connectivity is intermittent.

SIMULATION ONLY / NO REAL FUNDS

Concept, workflow, technical approach, and prototype limits

FACE-TO-FACE

Cached request QR and a locally checked confirmation code.

FAR RECIPIENT

Queue on the sender device, then synchronize when the server is reachable.

Connectivity gaps change the interaction

NetBridge studies two separate needs: a nearby handoff and a far-recipient send that can wait for reconnection.

How can a payment intent remain clear when the network is unavailable?

Design goal

Make request details, customer approval, merchant confirmation, and offline status visible to both roles.

Boundary

Demonstrate a fictional workflow in a PWA. No bank, UPI rail, or real-money settlement is connected.

Two scenarios use different paths

The face-to-face demo works as a local handoff after caching. Far-recipient delivery depends on later server synchronization.

FACE-TO-FACE / OFFLINE HANDOFF

Request QR -> approval -> typed code

01 Merchant creates a time-limited request QR.

02 Customer scans, reviews amount, and approves with demo PIN and Authenticator.

03 Customer displays a 10-character code; merchant types it and records the simulation.

FAR RECIPIENT / DEFERRED SYNC

Recipient ID -> queue -> sync

01 Sender enters a NetBridge ID and amount, without scanning a QR.

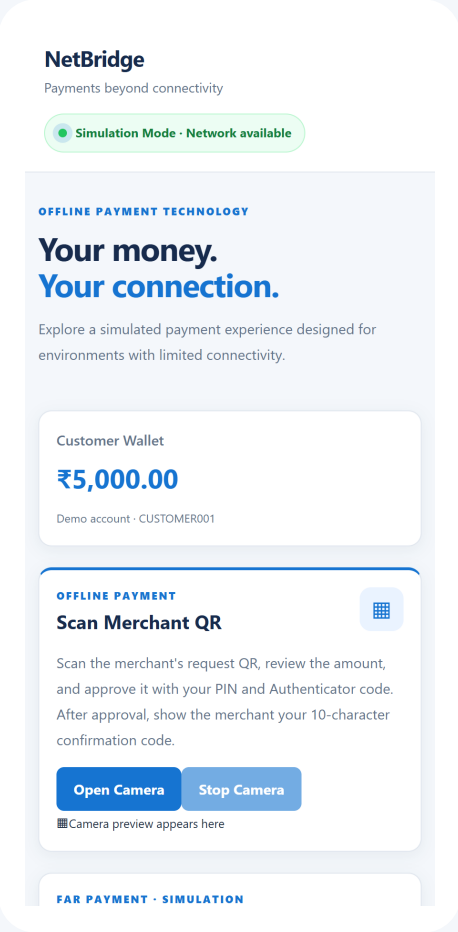
02 Offline send stays pending on the sender device.

03 After reconnecting, the server can deliver the simulated transaction to the recipient.

The far-recipient receiver needs the server to receive the simulated payment.

A mobile-first scan and approval journey

Current customer page preview. Values and activity shown in the image are sample data.



SCAN THE MERCHANT REQUEST

The customer reviews the request, then uses the demo PIN and Authenticator approval.

SHOW A SHORT CONFIRMATION CODE

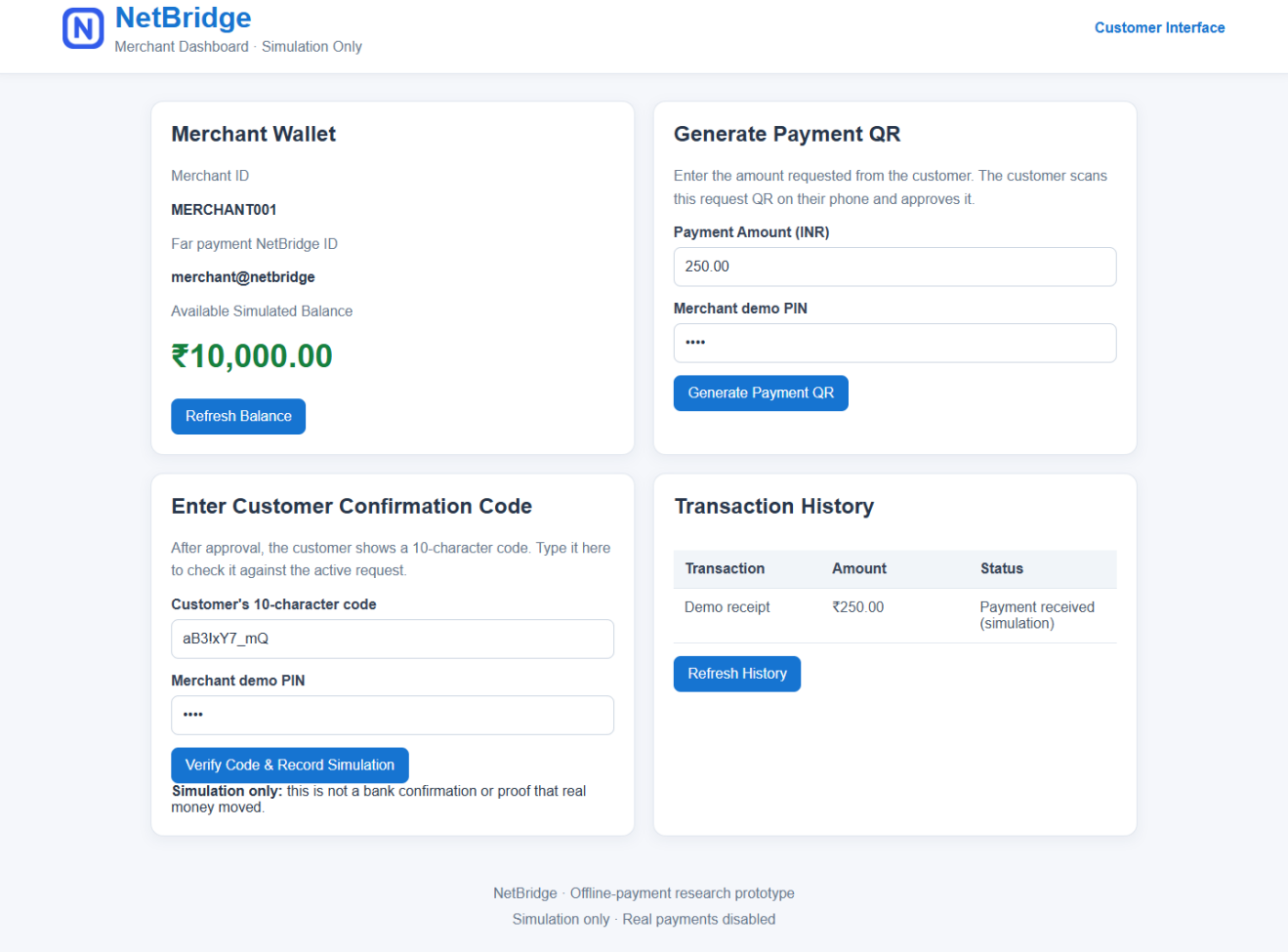
The customer displays a 10-character mixed-case code for the merchant to enter.

OR ENTER A FAR-RECIPIENT ID

Recipient ID and amount can remain pending locally until the sender reconnects to the server.

Create the request; verify the typed code

The merchant dashboard does not scan the customer confirmation code.



Merchant sequence

1. Enter amount and demo PIN.
2. Display the request QR.
3. Type the customer confirmation code.
4. Check it against the active request.
5. Record the simulation locally.

Representative current-page render. Sample values are not live transactions.

Browser clients keep the two paths distinct

The app shell supports the offline handoff. The server handles enrollment and far-payment synchronization.

CUSTOMER PWA

- Responsive customer web UI
- QR camera reader and Authenticator screens
- Service Worker and local demo state

MERCHANT DASHBOARD

- Request QR generation
- Typed confirmation-code entry
- Local receipt and history view

FLASK + SQLITE

- Authenticator and demo API routes
- Server-side demo ledger
- Far-payment synchronization

OFFLINE FACE-TO-FACE

Cached pages + QR and code handoff. No server round-trip is needed during the exchange.

FAR-RECIPIENT

Local pending send -> sender reconnects -> server sync -> recipient sees demo delivery.

Caching does not replace the Flask server for Authenticator setup or far-payment delivery.

Authorization is explicit at every step

The QR starts a request. The short code is a local matching step, not a bank confirmation.

01

Request

Merchant enters the amount and creates a request QR.

02

Review

Customer scans and checks merchant and amount.

03

Approve

Customer enters demo PIN and current Authenticator code.

04

Show code

Customer displays a 10-character confirmation code.

05

Record

Merchant types the code and records the simulation.

DEVICE RECORDS STAY SEPARATE

The customer cannot see merchant acceptance without a return communication path. The offline demo records a local simulation on each device.

What the prototype demonstrates - and what it does not

DEMONSTRATES

- Request QR matched to an active request
- Demo Authenticator code checks
- Same-device one-time request handling
- Offline client shell after app caching
- Far-payment queue and later server sync as a simulation

DOES NOT PROVE

- Real funds, UPI, bank rails, or settlement
- Bank-grade protection against a compromised phone
- Global double-spend prevention across offline copies
- Far-recipient delivery without a reachable server
- Customer identity or real payment confirmation

Use: "Payment recorded in the simulation."

A clear prototype claim with measurable checks

Suggested demonstration plan. These checks are not presented as results from a new test run.

CHECK THE CORE JOURNEYS

- Complete the face-to-face handoff with cached pages and network disabled.
- Confirm incorrect, expired, and reused codes are rejected.
- Queue a far payment offline, reconnect the sender, and confirm recipient-side delivery.
- Review status wording, recovery behavior, and local record consistency.

PROJECT CLAIM

A prototype of offline authorization and deferred demo synchronization - not a payment network.